

## **Carátula**

Título de trabajo: Aplicaciones de TADs

Integrantes del equipo:

- Trossero Mateo Ignacio
- Bay Gabriel

Fecha de entrega: 28/09/2026

Link de Repositorio: <https://github.com/Gallean27/AyED-2026c1-Bay-Trossero-Lopez.git>

## INTRODUCCIÓN

El presente Trabajo Práctico tiene como objetivo principal el diseño, implementación y evaluación experimental de estructuras de datos lineales y algoritmos de ordenamiento mediante el lenguaje de programación Python. A través de un enfoque práctico, se busca resolver problemas clásicos de la computación mediante el uso de Programación Orientada a Objetos (POO), pruebas unitarias automatizadas y análisis empírico de rendimiento.

Para responder a los requerimientos planteados, el trabajo aborda tres problemas concretos:

1. **Gestión eficiente de estructuras lineales:** La implementación de Tipos de Datos Abstractos (TDA) dinámicos y acotados, especificando el comportamiento de listas enlazadas y colas circulares basadas en arreglos de tamaño fijo.
2. **Simulación de gestión de procesos:** La resolución del problema de planificación de CPU en un sistema operativo mediante la simulación del algoritmo *Round Robin*, empleando la cola circular desarrollada para gestionar el orden de ejecución y calcular métricas operativas clave (tiempo de retorno y tiempo de espera).
3. **Análisis empírico de eficiencia algorítmica:** La comparación del rendimiento de diferentes algoritmos de ordenamiento (*Burbuja*, *Quicksort*, *Radix Sort* y el algoritmo *Timsort* nativo de Python) al procesar conjuntos de datos aleatorios de tamaño variable.

A lo largo del informe, el lector encontrará la justificación teórica de los métodos implementados —aplicando conceptos de complejidad temporal y espacial mediante la notación Ómicron Grande ( $O$ )—, los diagramas y códigos correspondientes a cada proyecto, junto con gráficos comparativos que contrastan los tiempos reales de ejecución frente a las cotas teóricas de rendimiento.

## DESARROLLO DE LOS PROBLEMAS

### 3.1. Problema N° 1: Implementación de Lista Doblemente Enlazada (TDA)

#### A. Análisis y planteo

El problema consiste en abstraer e implementar una estructura de datos lineal secuencial capaz de gestionar una colección dinámica de elementos con acceso bidireccional. A diferencia de un arreglo estático, no se requiere memoria contigua preasignada, evitando el costo de relocalización o redimensionamiento.

Para esta estructura, las operaciones identificadas como críticas son:

- **Inserción y Eliminación en los extremos (cabeza y cola):** Deben realizarse en tiempo constante, evitando recorrer la estructura completa.
- **Recorrido bidireccional (hacia adelante y hacia atrás):** Permite inspeccionar los nodos en ambos sentidos de manera eficiente.
- **Eliminación por valor o posición intermedia:** Requiere una búsqueda lineal previa y la posterior reestructuración de enlaces.

#### Selección de la estructura:

Se seleccionó una **Lista Doblemente Enlazada (Doubly Linked List)**. Cada nodo almacena el dato deseado y dos punteros o referencias: uno al nodo siguiente (**siguiente**) y otro al anterior (**anterior**). La lista mantiene punteros explícitos a la cabeza (**cabeza**) y a la cola (**cola**), además de un contador de tamaño (**tamano**).

#### Supuestos y razonamiento de la elección:

1. **Dinamicidad:** Es ideal cuando el número total de elementos es indeterminado o varía frecuentemente durante la ejecución del programa.
2. **Uso típico vs. Peor caso:** Las inserciones/eliminaciones en los extremos son  $O(1)$  en todo momento. El peor caso ocurre durante la búsqueda o eliminación de un nodo intermedio, que requiere un recorrido de orden  $O(n)$ .
3. **Compromiso de memoria:** Se acepta un sobre costo leve de memoria por el almacenamiento de los dos punteros por nodo a cambio del beneficio de poder navegar hacia atrás sin tener que reiniciar el recorrido desde la cabeza.